**Project Analysis Report**

# CloudNote AI: Cloud-Based Intelligent Video Note Extraction and Semantic Search Platform

Engineering Capstone Project - 1 (23IE4053R/23IE4053A)


**Submitted by**

K. Mithilesh Reddy (2320030330)

Nitin Tulugu (2320030186)

Sargu Sai Chetan (2320030348)

**Under the guidance of**

**Dr. Neelima Gogineni**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**KONERU LAKSHMAIAH EDUCATION FOUNDATION (KL University)**

Hyderabad, India


Academic Year 2026-2027

# TABLE OF CONTENTS


**1. Purpose and Summary** ..... **1**

1.1 Purpose ..... 1

1.2 Summary of Findings ..... 1

**2. Problem and Requirement Analysis** ..... **2**

2.1 Stakeholders ..... 2

2.2 Requirement Coverage ..... 2

**3. Feasibility Analysis** ..... **3**

3.1 Technical Feasibility ..... 3

3.2 Economic Feasibility ..... 3

3.3 Operational Feasibility ..... 3

3.4 Schedule Feasibility ..... 3

3.5 Legal and Ethical Feasibility ..... 3

**4. Cost Analysis** ..... **4**

4.1 Actual Spend ..... 4

4.2 Cost Drivers ..... 4

4.3 Scenario Estimates ..... 4

**5. Performance Analysis** ..... **6**

5.1 Measured Latency ..... 6

5.2 Bottleneck Analysis ..... 6

5.3 Scalability ..... 6

**6. Quality Analysis** ..... **7**

6.1 Testing ..... 7

6.2 Defect Analysis ..... 7

6.3 Code Size ..... 7

**7. Risk Analysis** ..... **8**

**8. SWOT Analysis** ..... **9**

**9. Design Decision Analysis** ..... **10**

**10. Limitations and Recommendations** ..... **11**

10.1 Limitations ..... 11

10.2 Recommendations ..... 11

# CHAPTER 1
# PURPOSE AND SUMMARY

## 1.1 Purpose

This report analyses CloudNote AI from the perspectives that matter for deciding whether, and how, it could be used beyond the capstone: feasibility, cost, performance, quality, risk and the main design decisions. It complements the Final Project Report, which describes what was built and how.

## 1.2 Summary of Findings

- **Technically feasible and demonstrated:** the complete pipeline runs on real AWS services and was verified end to end on 29 October 2026.
- **Economically feasible at demo scale:** Cost Explorer showed no measurable charge for October 2026 (data may lag by about a day). At class scale, Amazon Transcribe dominates cost, so transcript import and usage limits matter.
- **Performance:** interactive operations complete in about 1 to 5 seconds (medians); long tails come from third-party model providers.
- **Quality:** 59 automated tests pass; six deployment defects were found and fixed; the browser run showed no console errors.
- **Main risks:** dependence on free-tier model providers while Bedrock is blocked, open sign-up, and unmeasured accuracy of reference recovery on real lecture series.

# CHAPTER 2
# PROBLEM AND REQUIREMENT ANALYSIS

## 2.1 Stakeholders

**Table 1: Stakeholders and their needs**

| Stakeholder | Needs |
|---|---|
| Students | Find explanations quickly, review lectures efficiently, trust answers, keep data private. |
| Instructors | Help students connect lectures in a series; understand which topics are assumed but never explained. |
| Department / institution | Low cost, no servers to maintain, secure handling of recordings. |
| Project team | A system that can be built, deployed and evaluated within one academic term. |

## 2.2 Requirement Coverage

All 13 functional requirements in the Final Project Report are implemented and were exercised on the live deployment. One quality requirement is only partly met: semantic search is implemented but disabled, because Bedrock embeddings are unavailable on the project's AWS account.

# CHAPTER 3
# FEASIBILITY ANALYSIS

## 3.1 Technical Feasibility

Every component uses a managed service with documented APIs: Transcribe for speech, Cognito for identity, Lambda and API Gateway for compute, DynamoDB for storage and S3 with CloudFront for files. The team built, deployed and verified the full system, which demonstrates technical feasibility directly. The one technical constraint encountered was account-level: the Bedrock quota was zero, which was handled with a provider fallback chain.

## 3.2 Economic Feasibility

The serverless design has no fixed monthly server cost; services are billed per use, and most have free allowances at small scale. Chapter 4 analyses costs in detail. At the scale of a demonstration or a small class pilot the cost is low; at full institutional scale, transcription cost requires planning.

## 3.3 Operational Feasibility

The application runs in any modern browser, including phones, and needs no installation. Users sign up with an email address. A user manual with screenshots is provided. Deployment and removal are single commands, and the whole stack is defined in one template, so a new maintainer can redeploy it.

## 3.4 Schedule Feasibility

The core pipeline and all planned features were completed within the capstone term. The project plan estimated the original extensions (Context Recovery 3 to 4 days, Notes vs Video 2 to 3 days, Remember the Missing Explanation 1 to 2 days) on top of a working core; these were delivered, together with additional features such as Know more and the library map.

## 3.5 Legal and Ethical Feasibility

Users upload their own recordings and are responsible for having the right to do so. Predictions concern content, not people. Generated content is labelled by source. The main ethical concern is that, while a third-party provider is used, transcript text leaves AWS; this is disclosed, and users are advised not to upload confidential recordings.

# CHAPTER 4
# COST ANALYSIS

## 4.1 Actual Spend

AWS Cost Explorer reported no measurable charge for the account in October 2026 up to 29 October (S3 and data transfer at 0.00 USD; Cost Explorer data can lag by about 24 hours). Development, deployment, several test accounts, several short Transcribe jobs and repeated benchmark runs therefore stayed within free allowances and negligible usage. The account holds 138.79 USD of promotional credits, which are applied before any charge.

## 4.2 Cost Drivers

**Table 2: Cost drivers by service**

| Service | Billing basis | Expected impact |
|---|---|---|
| Amazon Transcribe | Per second of audio (standard batch about 0.024 USD per minute in us-east-1 at the time of writing) | Dominant cost; scales with hours uploaded |
| Amazon Bedrock (when enabled) | Per token (Nova Lite is among the lowest-priced models) | Small: a few cents per hour of lecture |
| Groq / Gemini (current fallback) | Free tier with rate limits | No charge; reliability risk instead |
| Lambda | Per request and GB-second; monthly free allowance | Near zero at class scale |
| DynamoDB (on-demand) | Per request and stored GB | Near zero at class scale |
| S3 and CloudFront | Stored GB, requests, data transfer | Small; grows with stored video |
| Cognito | Per monthly active user, with a free allowance | Near zero for a class |

Prices change and differ by region; the AWS pricing pages should be checked before any budget decision.

## 4.3 Scenario Estimates

The table estimates Transcribe cost only, because it dominates. It assumes 0.024 USD per minute and that every recording is uploaded as media rather than imported as a transcript.

**Table 3: Estimated transcription cost by usage scenario**

| Scenario | Audio minutes | Approx. cost |
|---|---|---|
| Capstone demo: 4 videos of 5 minutes | 20 | 0.48 USD |
| One student, one course: 30 lectures of 50 minutes | 1,500 | 36 USD |
| Class of 60 students each uploading the same 30 lectures | 90,000 | 2,160 USD |
| Same class, lectures uploaded once and shared | 1,500 | 36 USD |

The analysis shows that the current per-user design is cheap for individuals but wasteful for classes, because every student would transcribe the same lecture. A shared course library, or importing existing captions as transcripts (which costs nothing to transcribe), would reduce class-scale cost by more than 95%. The per-user limits already in place (30 recordings, 500 MB per file) cap the worst case for any single account.

# CHAPTER 5
# PERFORMANCE ANALYSIS

## 5.1 Measured Latency

**Table 4: Observed end-to-end latency on the live deployment (29 October 2026)**

| Operation | Runs | Median | Range |
|---|---|---|---|
| Transcript import to AI notes | 3 | 4.6 s | 4.4 to 4.9 s |
| Upload, Transcribe and AI notes (11-s clip) | 3 | 17.3 s | 8.4 to 21.2 s |
| Ask your notes | 10 | 2.5 s | 2.0 to 21.9 s |
| Know more explanation | 5 | 4.1 s | 2.3 to 15.0 s |
| Library search (lexical) | 10 | 1.0 s | 0.9 to 1.6 s |

## 5.2 Bottleneck Analysis

- **Transcription:** job scheduling dominates for short clips (8.4 to 21.2 s for 11 seconds of audio). For long lectures, total time grows with audio length; the interface shows progress and fills in when done.
- **Language models:** typical answers take about 2 to 3 s; the long tails (15.0 s and 21.9 s) occurred when the free Groq tier rate-limited requests and the chain moved to the next provider.
- **Search:** about 1 s including intercontinental network latency; the in-memory index per Lambda container is fast for libraries of a few hundred recordings.
- **Cold starts:** the first request to an idle function adds latency; at higher traffic this is amortised.

## 5.3 Scalability

All AWS components scale automatically. The API is throttled at about 25 requests per second with bursts of 50, which suits a class but would need raising for an institution. The limiting factors at scale are the per-container in-memory search index (suitable up to a few hundred recordings per user) and the free-tier model rate limits.

# CHAPTER 6
# QUALITY ANALYSIS

## 6.1 Testing

**Table 5: Automated test results**

| Suite | Tests | Passed |
|---|---|---|
| Engine | 13 | 13 |
| Backend core | 22 | 22 |
| API | 24 | 24 |
| AWS and Cognito (need local AWS mock) | - | Not run |

A browser-driven run on the live site covered every page, a real upload with Transcribe and the mobile layout, with no console errors.

## 6.2 Defect Analysis

Six defects appeared only on real AWS and were invisible locally: missing function code due to packaging, a missing database permission, the Free plan blocking Transcribe, the Bedrock quota, retired Gemini models, and unrecognised citation formats. All were fixed. The lesson is that local tests with in-memory providers verify logic but not cloud configuration; an automated post-deploy smoke test would catch the first two classes of defect immediately.

## 6.3 Code Size

**Table 6: Approximate lines of code**

| Area | Lines |
|---|---|
| Frontend JavaScript | 1,314 |
| Frontend CSS | 409 |
| Backend | 1,380 |
| Shared engine | 385 |
| Infrastructure | 382 |
| Tests | 1,008 |

# CHAPTER 7

# RISK ANALYSIS

**Table 7: Risk register**

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Free-tier model provider rate-limits or changes terms | High | Medium | Fallback chain to extractive notes; request Bedrock quota increase |
| Bedrock quota stays at zero | Medium | Medium | Groq fallback in place; AWS Support case |
| Open sign-up leads to unexpected Transcribe spend | Medium | High | Restrict sign-up by email domain; budget alert; per-user limits |
| Transcript text sent to third-party provider | High (while fallback used) | Medium | Disclosure; advise against confidential uploads; return to Bedrock |
| Reference recovery misses or mislinks on real speech | Medium | Medium | User confirmation before any link; labelled dataset and evaluation planned |
| Deployment access key misuse | Low | High | Delete key after deployment; no keys in repository |
| Media bucket CORS allows any origin | Low | Low | Uploads need presigned URLs; restrict origin to the site domain |
| No password reset or email verification | High | Low | Enable Cognito verification and recovery before wider use |

# CHAPTER 8
# SWOT ANALYSIS

**Table 8: SWOT analysis**

| Strengths | Weaknesses |
|---|---|
| Working, deployed, open-source system; serverless and low cost; every note and answer links to a moment; user-controlled linking; honest labelling of AI content. | Reference accuracy not yet measured on real lectures; English-only detection patterns; semantic search disabled; depends on free-tier providers today. |

**Table 9: SWOT analysis (continued)**

| Opportunities | Threats |
|---|---|
| Course-level shared libraries; instructor gap reports; multilingual support for Indian languages; integration with existing lecture platforms. | Provider pricing or quota changes; general-purpose AI tools adding similar features; privacy expectations around recordings. |

# CHAPTER 9

# DESIGN DECISION ANALYSIS

**Table 10: Key design decisions and alternatives**

| Decision | Chosen | Alternative | Reason |
|---|---|---|---|
| Compute | Serverless (Lambda) | Virtual machine (EC2) | No idle cost, no patching, automatic scaling |
| Database | DynamoDB single table | Relational database | Per-user partitioning, no server, pay per request |
| Search | TF-IDF, Titan when available | Vector database | No extra service at small scale; documented upgrade path |
| Reference detection | Hand-written patterns | Trained classifier | Explainable and needs no training data |
| Language model | Bedrock with fallbacks | Single provider | Resilience to quotas and outages |
| Frontend | Vanilla JavaScript | React or similar | Small bundle, no build complexity, full control |
| Linking | User confirmation | Automatic linking | Avoids silent mistakes; keeps the user in control |

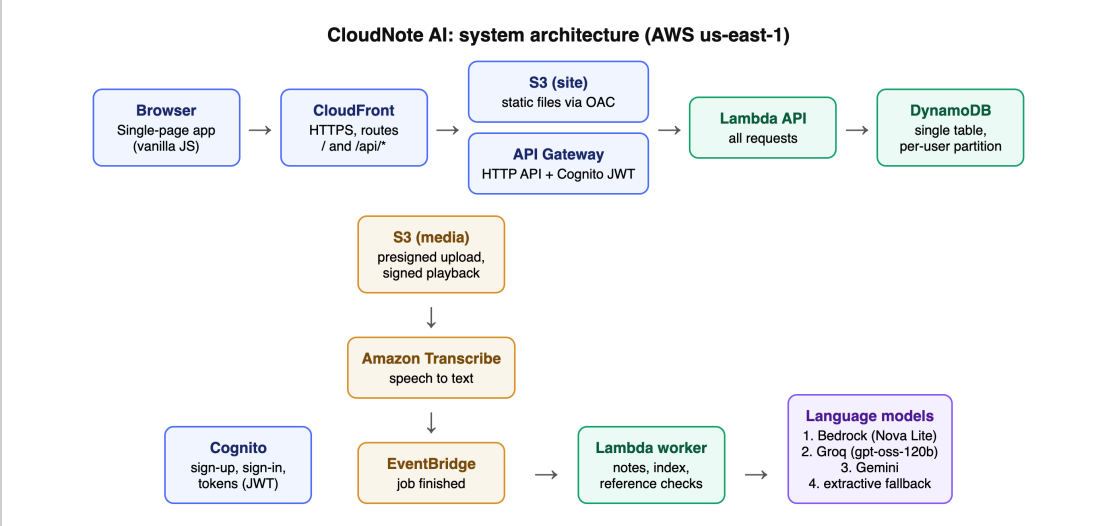



*Figure 1: Architecture resulting from these decisions*

# CHAPTER 10
# LIMITATIONS AND RECOMMENDATIONS

## 10.1 Limitations

- Reference recovery has been exercised on illustrative samples but not scored on real lecture series.
- Measurements use small samples from one day and one client location.
- Semantic search and Bedrock models have not run on the project account.

## 10.2 Recommendations

1. Before wider use: restrict sign-up, enable email verification and password recovery, set a budget alert, and delete the deployment access key.
2. Request a Bedrock quota increase, then enable Titan embeddings and tune thresholds on real data.
3. Record or collect a small set of real lecture series and measure detection recall and candidate precision.
4. For classes, add shared course libraries or caption import to avoid transcribing the same lecture many times.
5. Add an automated post-deploy smoke test that signs up a test user and exercises the main endpoints.